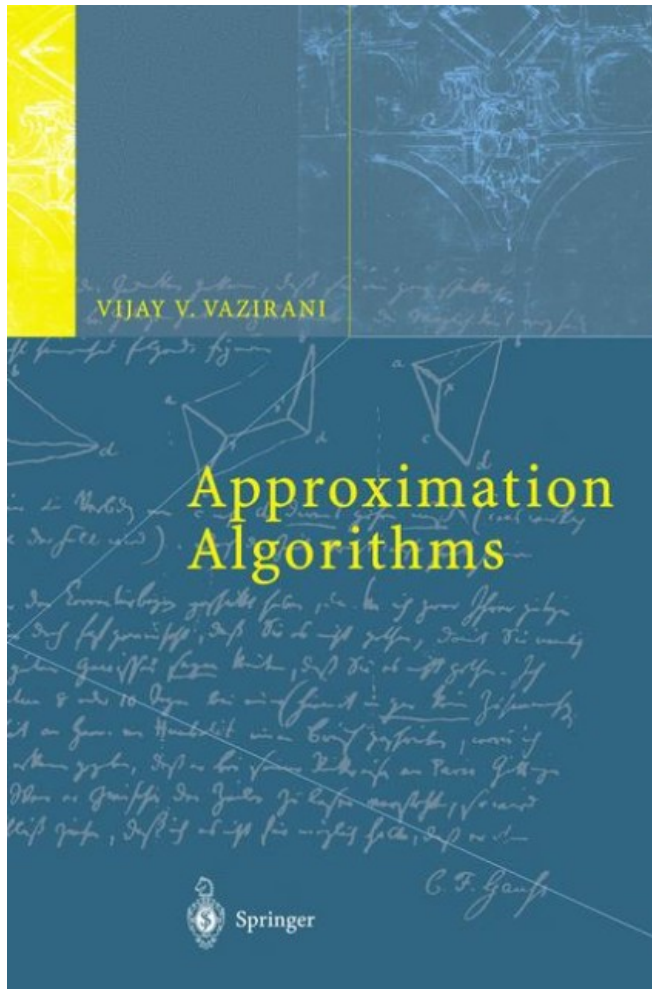


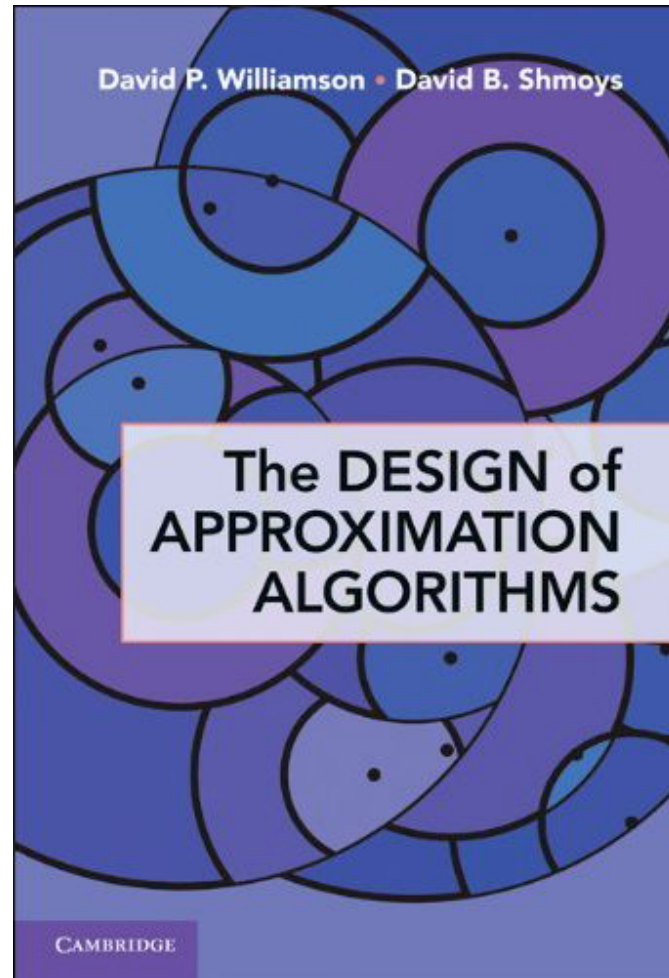
Complexity of Algorithms

Introduction to Approximation Algorithms

Textbooks



Vijay V. Vazirani:
Approximation Algorithms
Springer-Verlag, 2003.



D. P. Williamson & D. B. Shmoys:
The Design of Approximation Algorithms
Cambridge-Verlag, 2011.

<http://www.designofapproxalgs.com/> 

Approximation Algorithms

„All exact science is dominated by the idea of approximation.“



Bertrand Russell
(1872–1970)

Approximation Algorithms

- Many optimization problems are NP-hard!
(For example, the traveling salesperson problem.)
- $\rightsquigarrow$ an optimal solution cannot be efficiently computed unless $P=NP$.
- However, good approximate solutions can often be found efficiently!
- Will see two examples of such approximation algorithms for the Vertex Cover and Steiner Tree problems, respectively.

Complexity of Algorithms

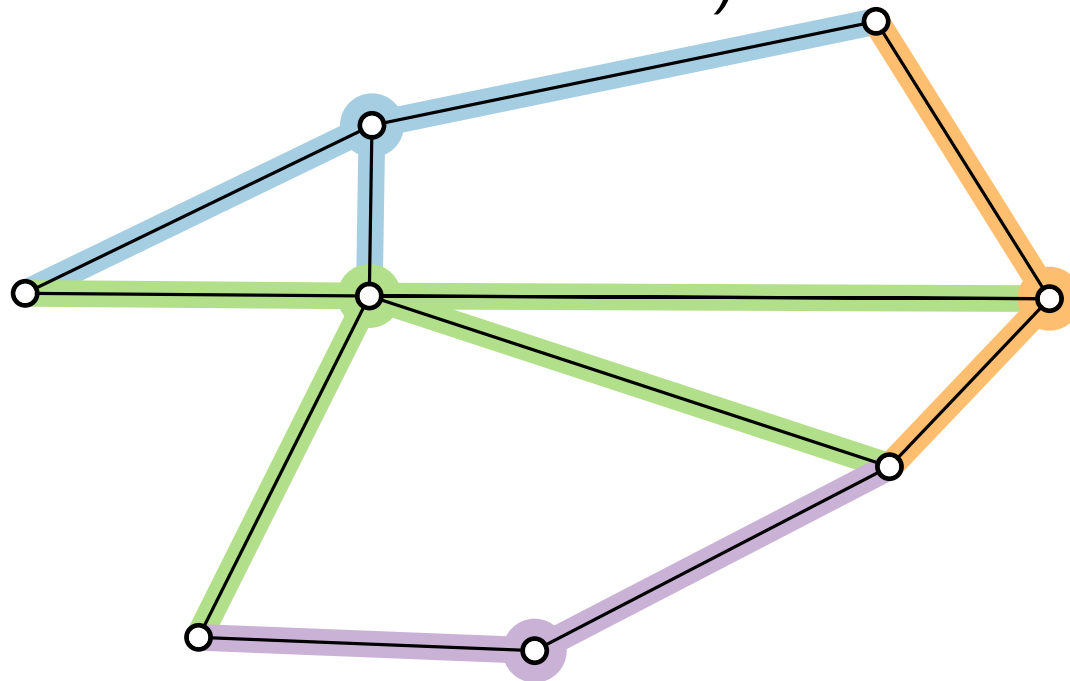
Introduction to Approximation Algorithms

Part I: Vertex Cover

VERTEXCOVER

Input: graph G

Output: a minimum **vertex cover**, that is, a smallest subset V' of vertices s. t. every edge is **covered** (formally, $V' \subseteq V(G)$ s.t. for every $uv \in E(G)$, it holds that $u \in V'$ or $v \in V'$).



Optimum ($\text{OPT} = 4$) – but in general NP-hard to find :-)

Complexity of Algorithms

Introduction to Approximation Algorithms

Part II: Definition Approximation Algorithm

Definition Approximation Algorithm

Let Π be a minimization problem and $\alpha \geq 1$.

A factor- α approximation algorithm for Π is an efficient (i. e., polynomial time) algorithm that computes, for **any** input I , a solution s such that

$$\frac{\overbrace{\text{value of } s}^{\text{ALG}}}{\underbrace{\text{value of optimal solution } s^* \text{ to } I}_{\text{OPT}}} \leq \alpha.$$

Definition Approximation Algorithm

Let Π be a ~~maximization problem~~ ~~minimization problem~~ and $0 < \alpha \leq 1$ ~~$\alpha \geq 1$~~ .

A factor- α approximation algorithm for Π is an efficient (i. e., polynomial time) algorithm that computes, for **any** input I , a solution s such that

$$\frac{\overbrace{\text{value of } s}^{\text{ALG}}}{\underbrace{\text{value of optimal solution } s^* \text{ to } I}_{\text{OPT}}} \geq \alpha.$$

Complexity of Algorithms

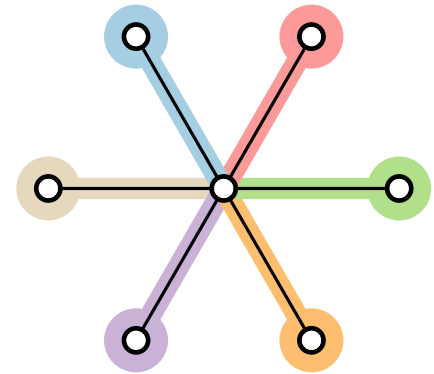
Introduction to Approximation Algorithms

Part III: Approximation Algorithm for Vertex Cover

Approximation Alg. for VERTEXCOVER

Ideas?

- Edge-Greedy
- Vertex-Greedy



Quality?

Problem: How can we estimate ALG/OPT – if it is hard to compute optimal solution?

Idea: Find a “good” lower bound $L \leq OPT$ for OPT and compare it to value ALG of our approximate solution.

$$\frac{ALG}{OPT} \leq \frac{ALG}{L}$$

Lower Bound by Matchings

Given a graph G , a set M of edges of G is a **matching** if no two edges of M are adjacent (i.e., share an end vertex).

M is **maximal** if there is no larger matching M' containing M as a subset.

$$OPT \geq |M|$$

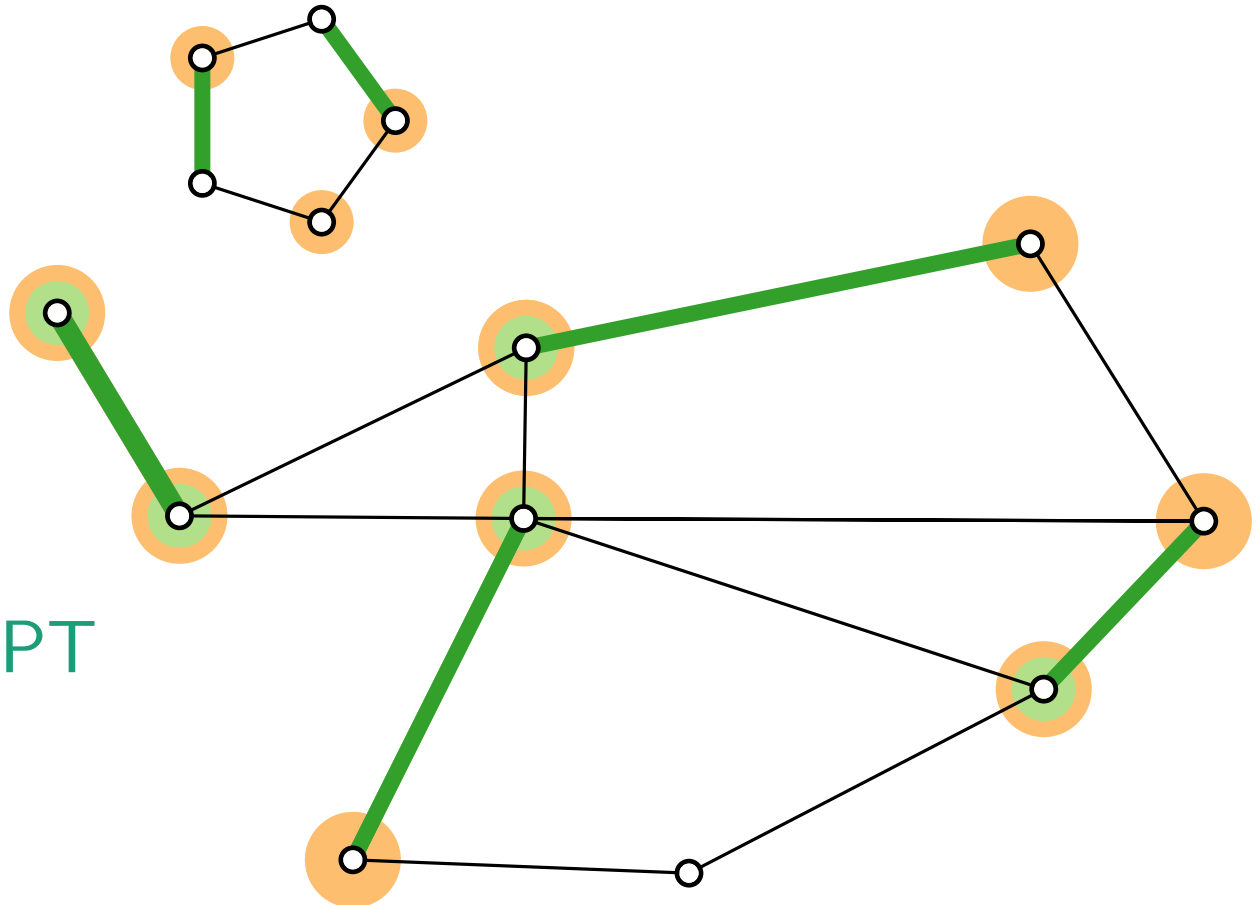
~~$$OPT = |M| ?$$~~

Vertex cover of M

Vertex cover of E !

$$ALG = 2 \cdot |M| \leq 2 \cdot OPT$$

$$\Rightarrow \frac{ALG}{OPT} \leq 2$$



Approximation Alg. for VERTEXCOVER

Algorithm VertexCover(G)

$M \leftarrow \emptyset$

foreach $e \in E(G)$ **do**

if e is not adjacent to any edge in M **then**
 $M \leftarrow M \cup \{e\}$

return $\{u, v \mid uv \in M\}$

Theorem. The above algorithm is a factor-2 approximation algorithm for VERTEXCOVER.

Proof. $ALG = 2 \cdot |M| \leq 2 \cdot OPT$



Approximability of VERTEX COVER

This simple algorithm gives the best known (constant) approximation factor for VERTEXCOVER.

If $P \neq NP$, VERTEXCOVER cannot be approximated within a factor of 1.3606.

VERTEXCOVER cannot be approximated within a factor of $2 - \epsilon$ for arbitrarily small $\epsilon > 0$ —if the so-called *Unique Games Conjecture* holds.

Complexity of Algorithms

Introduction to Approximation Algorithms

Part IV: Steiner Tree

STEINERTREE

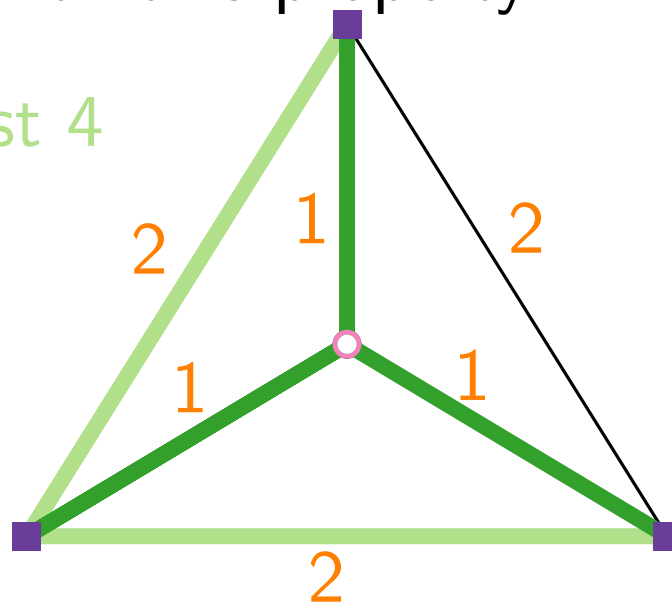
Given: A graph G with **edge weight** $c(e)$ for every edge e and a partition of its vertices into a set T of **terminals** and a set S of **Steiner vertices**.

Find: A **subtree** B of G that

- contains all **terminals** (i.e., $T \subseteq V(B)$) and
- has **minimum cost** $c(B) := \sum_{e \text{ is edge in } B} c(e)$ among all subtrees with this property.

feasible solution with cost 4

optimum solution
with cost 3



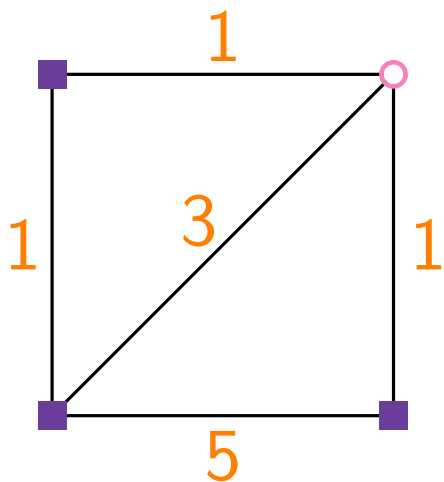
■ terminal

○ Steiner vertex

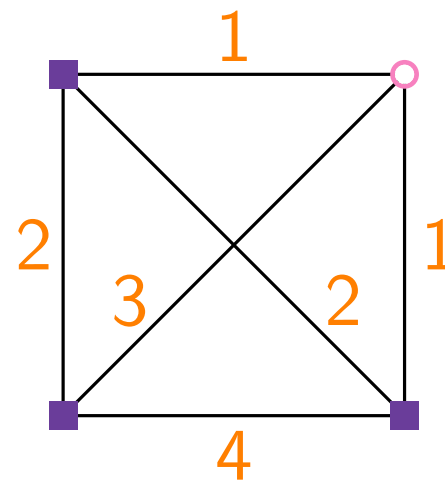
METRICSTEINERTREE

Restriction of STEINERTREE where the graph G is complete (i.e., has an edge uv for every pair of distinct vertices u, v) and the cost function is **metric**, i.e., for every triple u, v, w of vertices, we have $c(u, w) \leq c(u, v) + c(v, w)$.

It can be proven (not in this lecture) that it suffices to design an approximation algorithm for METRICSTEINERTREE



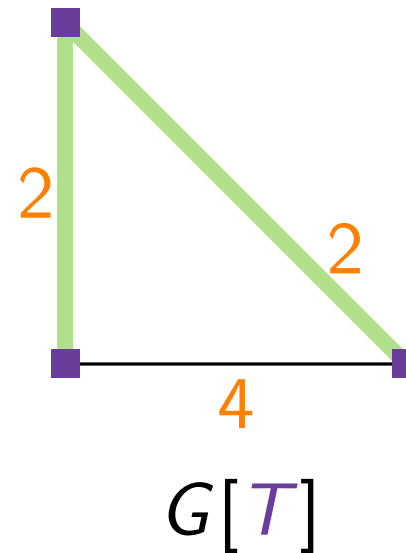
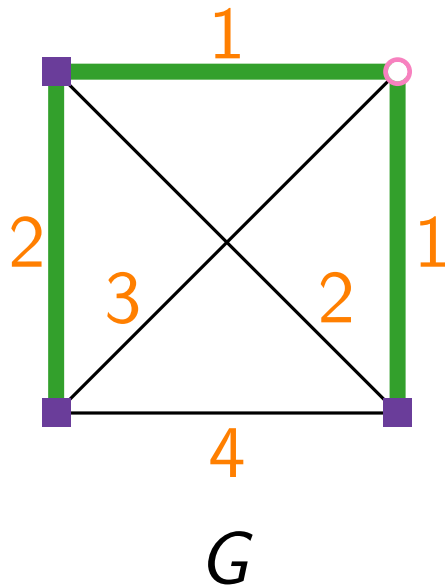
- not complete
- not metric



- complete
- metric

2-Approximation for STEINERTREE

Theorem. For an instance of METRICSTEINERTREE, let B be a minimum spanning tree (MST) of the subgraph $G[T]$ induced by the terminal set T . Then $c(B) \leq 2 \cdot \text{OPT}$.



Proof of Approximation Factor

Consider an optimal Steiner tree B^* .

Duplicate all edges of B^* . $\Rightarrow$ graph B' with cost $c(B') = 2 \cdot \text{OPT}$.

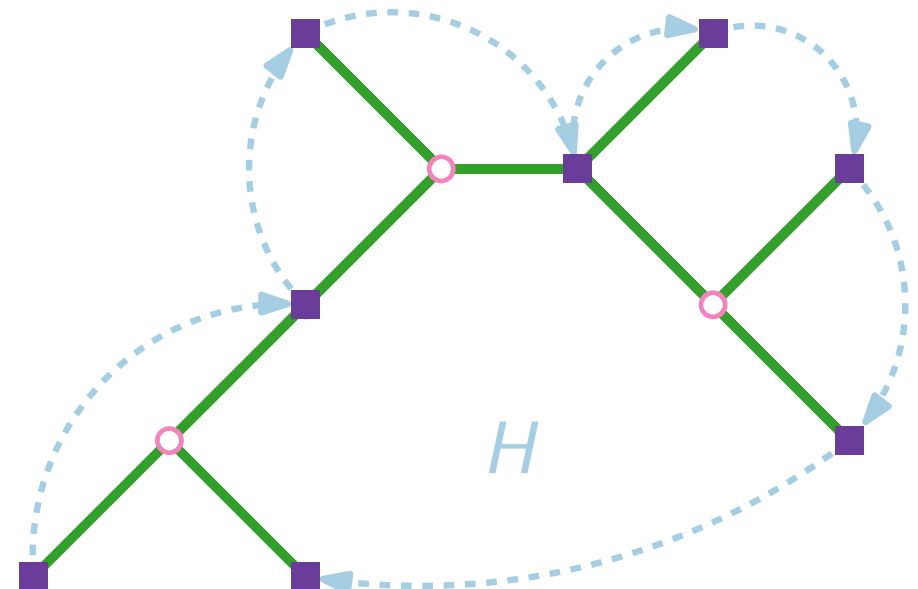
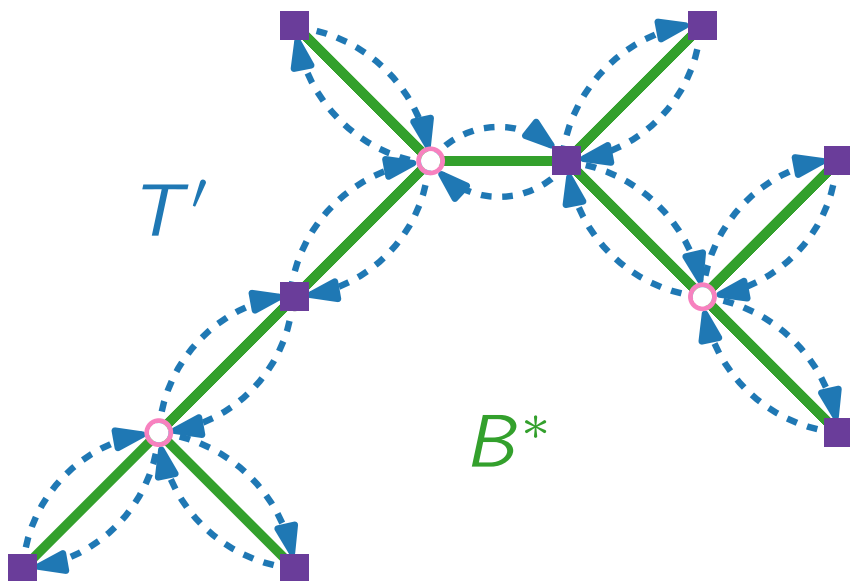
Find tour T' in B' traversing every edge exactly once (DFS in B^*)

$$\Rightarrow c(T') = c(B') = 2 \cdot \text{OPT}$$

Find path H in $G[T]$ that visits all terminals by “short-cutting” Steiner vertices and previously visited terminals.

$\Rightarrow c(H) \leq c(T') = 2 \cdot \text{OPT}$ since G is metric.

MST B of $G[T]$ costs $c(B) \leq c(H) \leq 2 \cdot \text{OPT}$
since H is a spanning tree of $G[T]$.



Outlook

Today, approximation algorithms are known for hundreds of optimisation problems arising in various areas of application such as logistics, network design, scheduling, data mining, and machine learning.

It is a vibrant research area with many beautiful ideas in algorithms design and analysis

If you are interested in this topic, for example, as part of your third-year project then please contact me (Joachim Spoerhase; joachim.spoerhase@liverpool.ac.uk).